

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

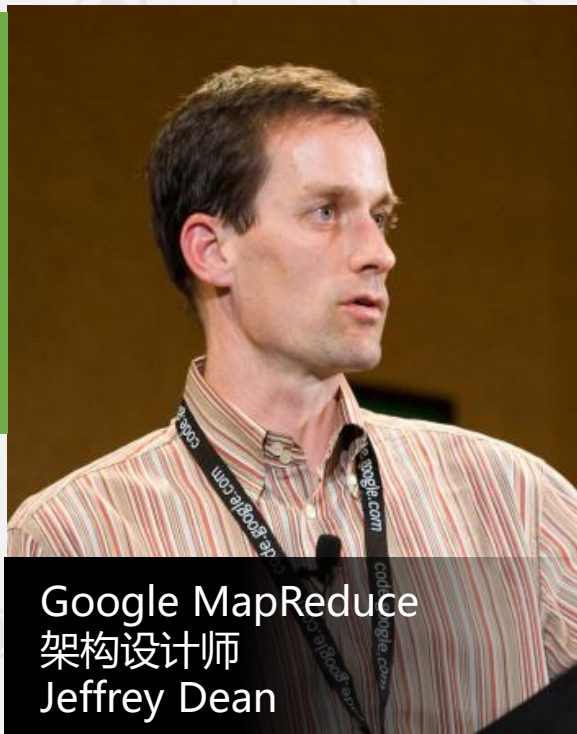
2.9 Google应用程序引擎

2.2分布式数据处理MapReduce

- ▶ 2.2.1 产生背景
- 2.2.2 编程模型
- 2.2.3 实现机制
- 2.2.4 案例分析

2.2 分布式数据处理 MapReduce

● 产生背景



Jeffery Dean, 美国计算机科学家和软件工程师, 出生于1968年, 毕业于华盛顿大学, 2009年当选美国工程院院士, 主要作品: MapReduce、BigTable、Spanner、TensorFlow等。

- Compilers don't warn Jeff Dean. Jeff Dean warns compilers.
- gcc -O4 emails your code to Jeff Dean for a rewrite.
- When Jeff has trouble sleeping, he Mapreduces sheep.

2.2 分布式数据处理 MapReduce

- 产生背景



MapReduce这种**并行编程模式**思想最早是在**1995年**提出的。

与传统的分布式程序设计相比，MapReduce封装了**并行处理、容错处理、本地化计算、负载均衡**等细节，还提供了一个**简单而强大的接口**。

MapReduce把对数据集的大规模操作，**分发给一个主节点管理下的各分节点共同完成，通过这种方式实现任务的可靠执行与容错机制**。

2.2分布式数据处理MapReduce

2.2.1 产生背景

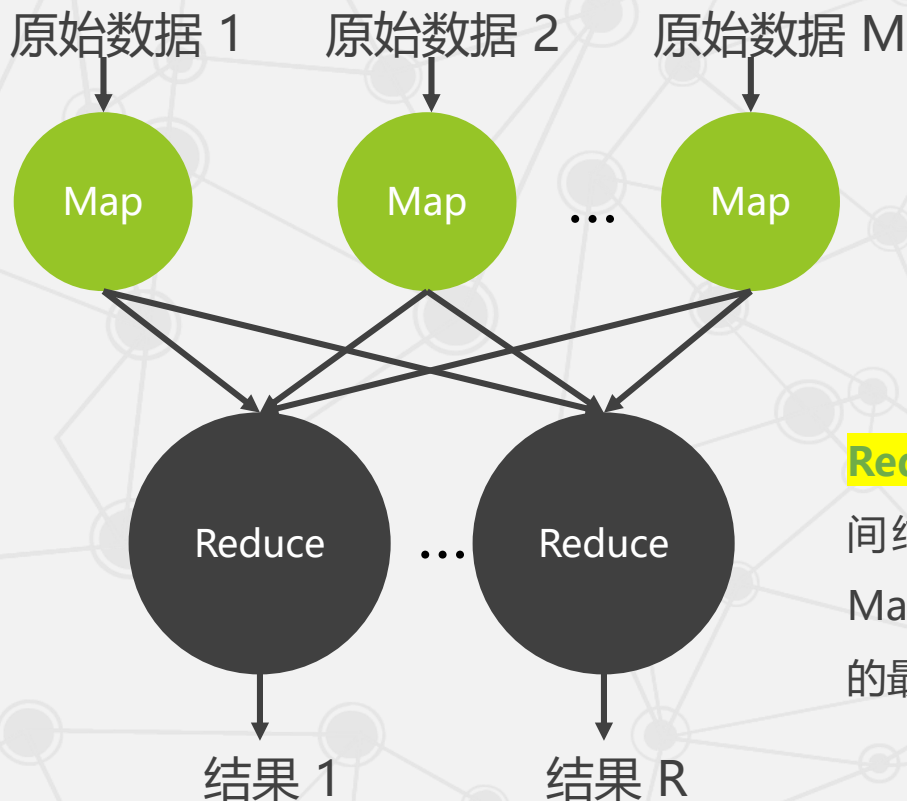
▶ 2.2.2 编程模型

2.2.3 实现机制

2.2.4 案例分析

2.2 分布式数据处理 MapReduce

● 编程模型



Map函数——对一部分原始数据进行指定的操作。每个Map操作都针对**不同**的原始数据，因此，Map与Map之间是互相**独立**的，这使得它们可以充分**并行化**。

Reduce操作——对每个Map所产生的一部分中间结果进行**合并**操作，每个Reduce所处理的Map中间结果是**互不交叉**的，所有Reduce产生的最终结果经过简单连接就形成了完整的结果集。

2.2 分布式数据处理 MapReduce

• 编程模型

Map: $(in_key, in_value) \rightarrow \{(key_j, value_j) \mid j = 1 \dots k\}$

Reduce: $(key, [value_1, \dots, value_m]) \rightarrow (key, final_value)$

Map输入参数: in_key 和 in_value , 它指明了Map需要处理的原始数据

Map输出结果: 一组 $\langle key, value \rangle$ 对, 这是经过Map操作后所产生的中间结果

Reduce输入参数: $(key, [value_1, \dots, value_m])$

Reduce工作:
对这些对应相同key的value值进行**归并**处理

Reduce输出结果: **一对**
 $(key, final_value)$, 所有Reduce的结果并在一起就是最终结果

2.2 分布式数据处理 MapReduce

• 编程模型

Map: $(in_key, in_value) \rightarrow \{(key_j, value_j) \mid j = 1 \dots k\}$

Reduce: $(key, [value_1, \dots, value_m]) \rightarrow (key, final_value)$

Map输入参数: in_key 和 in_value , 它指明了Map需要处理的原始数据

Map输出结果: 一组
 $\langle key, value \rangle$ 对, 这是经过Map操作后所产生的中间结果

Reduce输入参数: $(key, [value_1, \dots, value_m])$

Reduce工作:
对这些对应相同key的value值进行归并处理

Reduce输出结果: 一对
 $(key, final_value)$, 所有Reduce的结果并在一起就是最终结果

2.2 分布式数据处理 MapReduce

• 编程模型

Map: $(in_key, in_value) \rightarrow \{(key_j, value_j) \mid j = 1 \dots k\}$

Reduce: $(key, [value_1, \dots, value_m]) \rightarrow (key, final_value)$

Map输入参数: in_key 和 in_value ，它指明了Map需要处理的原始数据

Map输出结果: 一组
 $\langle key, value \rangle$ 对，这是经过Map操作后所产生的中间结果

Reduce输入参数: $(key, [value_1, \dots, value_m])$

Reduce工作:
对这些对应相同key的value值进行归并处理

Reduce输出结果: 一对
 $(key, final_value)$ ，所有Reduce的结果并在一起就是最终结果

2.2 分布式数据处理 MapReduce

• 编程模型

Map: $(in_key, in_value) \rightarrow \{(key_j, value_j) \mid j = 1 \dots k\}$

Reduce: $(key, [value_1, \dots, value_m]) \rightarrow (key, final_value)$

Map输入参数: in_key 和 in_value , 它指明了Map需要处理的原始数据

Map输出结果: 一组
 $\langle key, value \rangle$ 对, 这是经过Map操作后所产生的中间结果

Reduce输入参数: $(key, [value_1, \dots, value_m])$

Reduce工作:
对这些对应相同key的value值进行归并处理

Reduce输出结果: 一对
 $(key, final_value)$, 所有Reduce的结果并在一起就是最终结果

2.2 分布式数据处理 MapReduce

- 编程模型

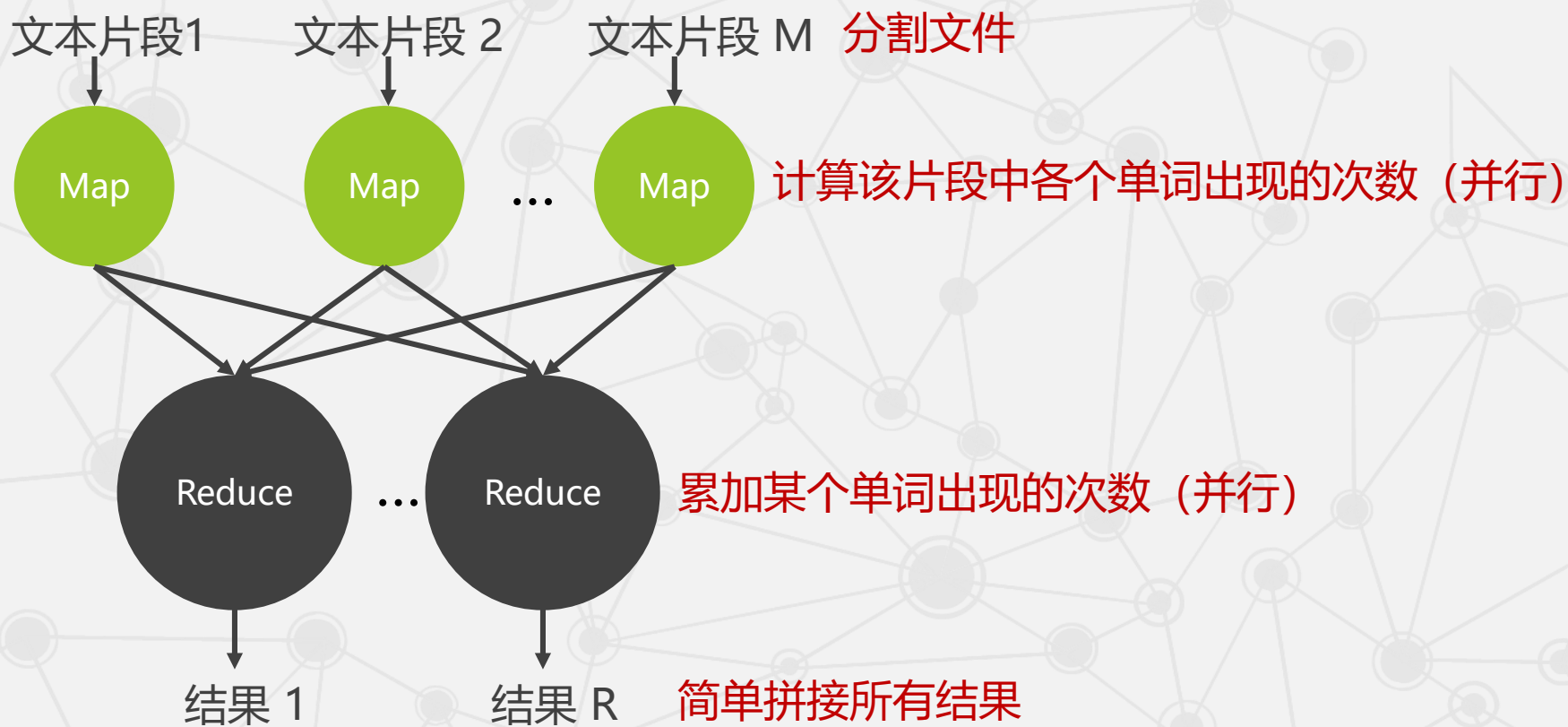
计算大型文本文件中各个单词出现的次数。

传统编程怎么做？

2.2 分布式数据处理 MapReduce

- 编程模型

计算大型文本文件中各个单词出现的次数。



2.2分布式数据处理MapReduce

2.2.1 产生背景

2.2.2 编程模型

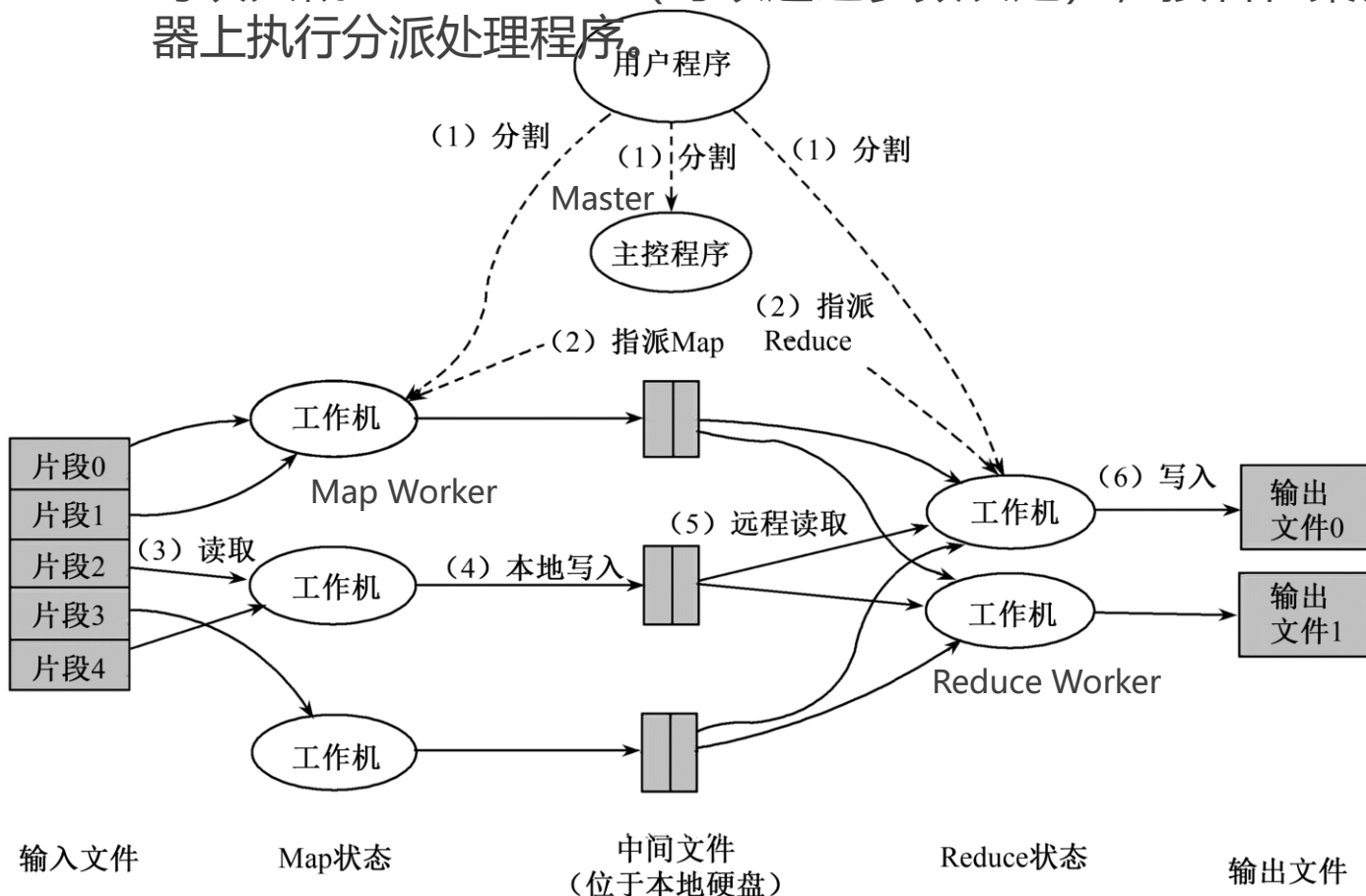
► 2.2.3 实现机制

2.2.4 案例分析

2.2 分布式数据处理MapReduce

● 实现机制

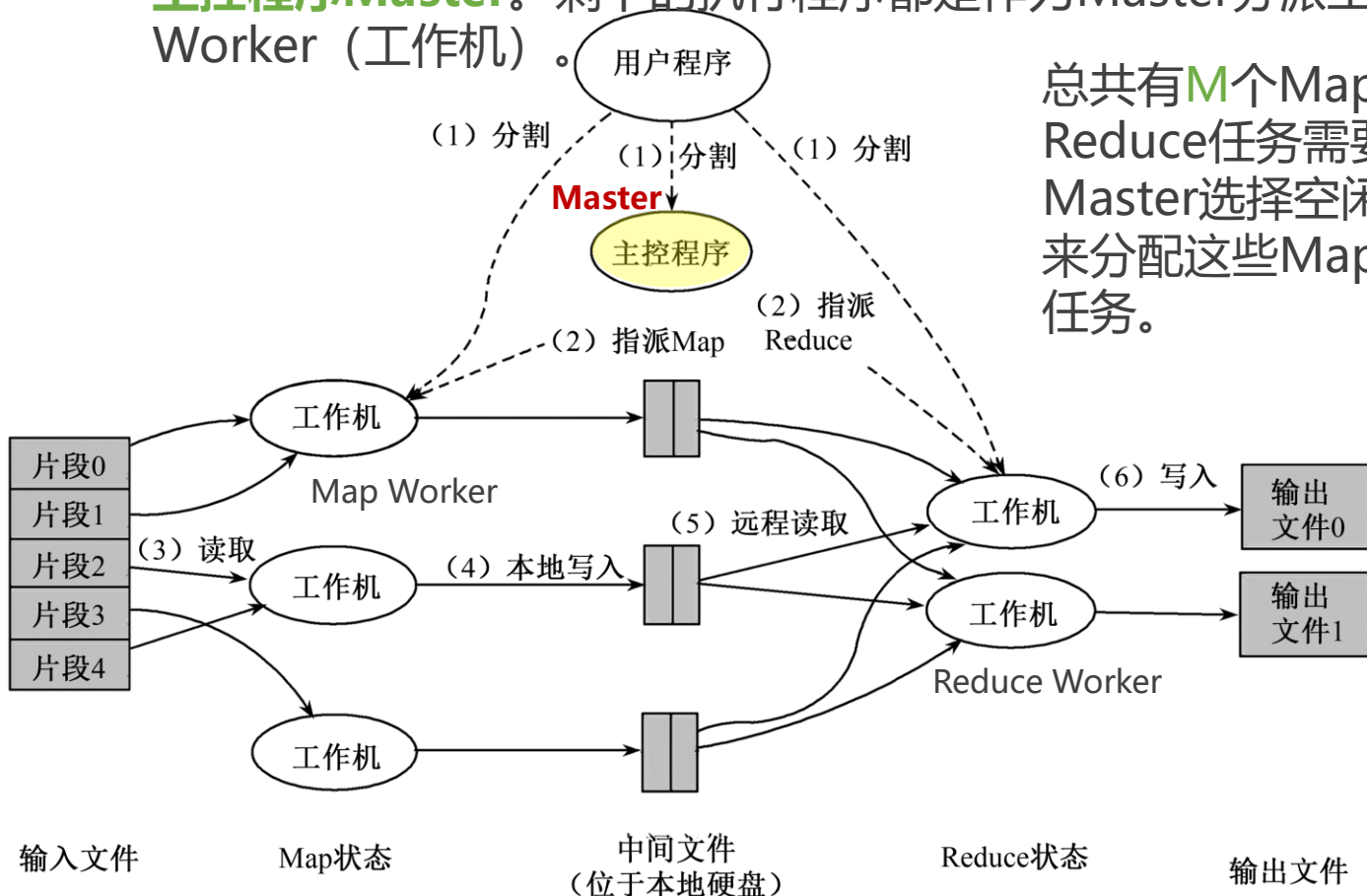
(1) **分割输入文件**。MapReduce函数首先把**输入文件分成M块**，每块大概16M~64MB（可以通过参数决定），接着在集群的机器上执行分派处理程序。



2.2 分布式数据处理MapReduce

● 实现机制

(2) **分派程序**。这些分派的执行程序中有一个程序比较特别，它是**主控程序Master**。剩下的执行程序都是作为Master分派工作的Worker（工作机）。



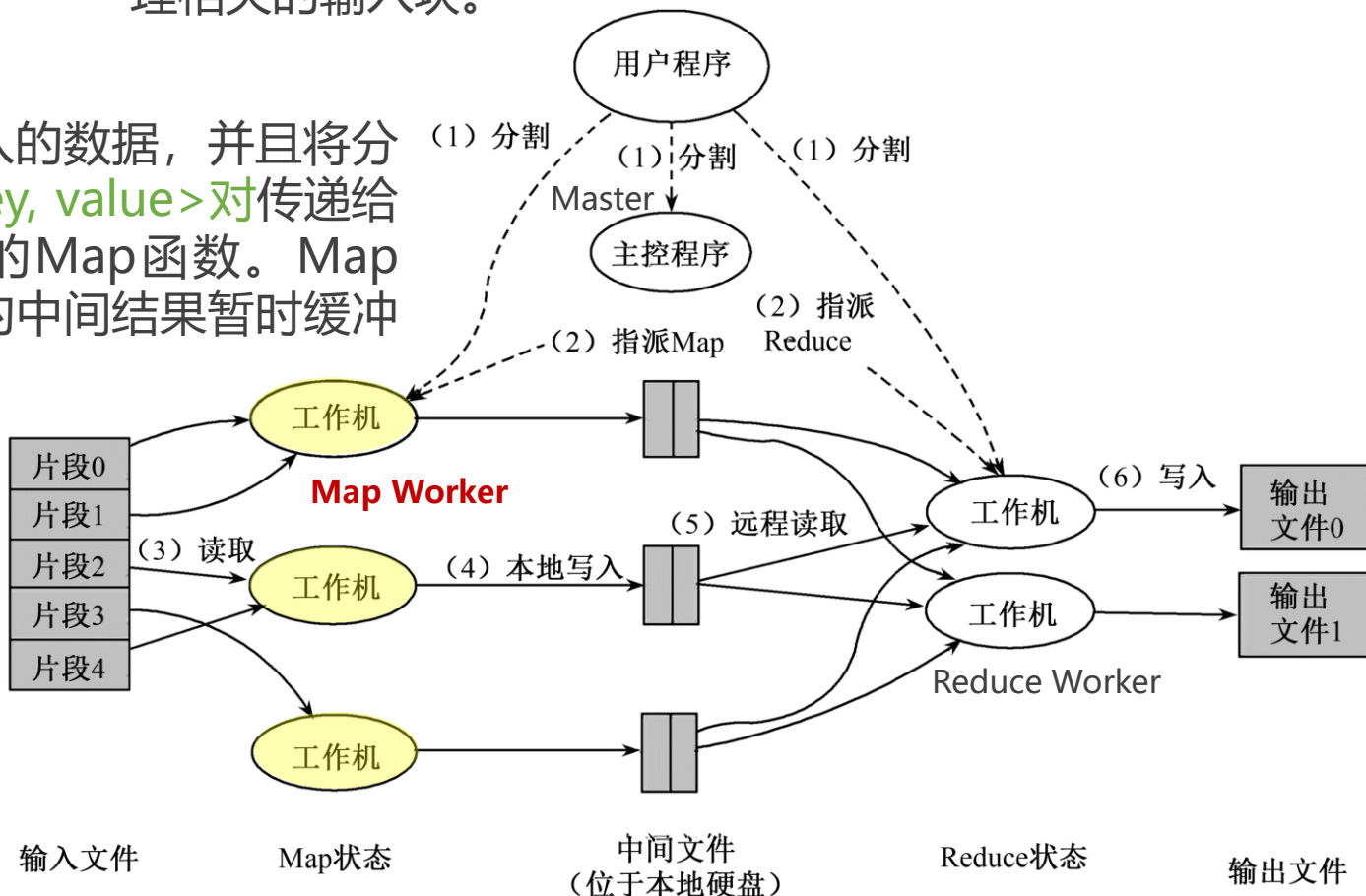
总共有M个Map任务和R个Reduce任务需要分派，Master选择空闲的Worker来分配这些Map或Reduce任务。

2.2 分布式数据处理MapReduce

● 实现机制

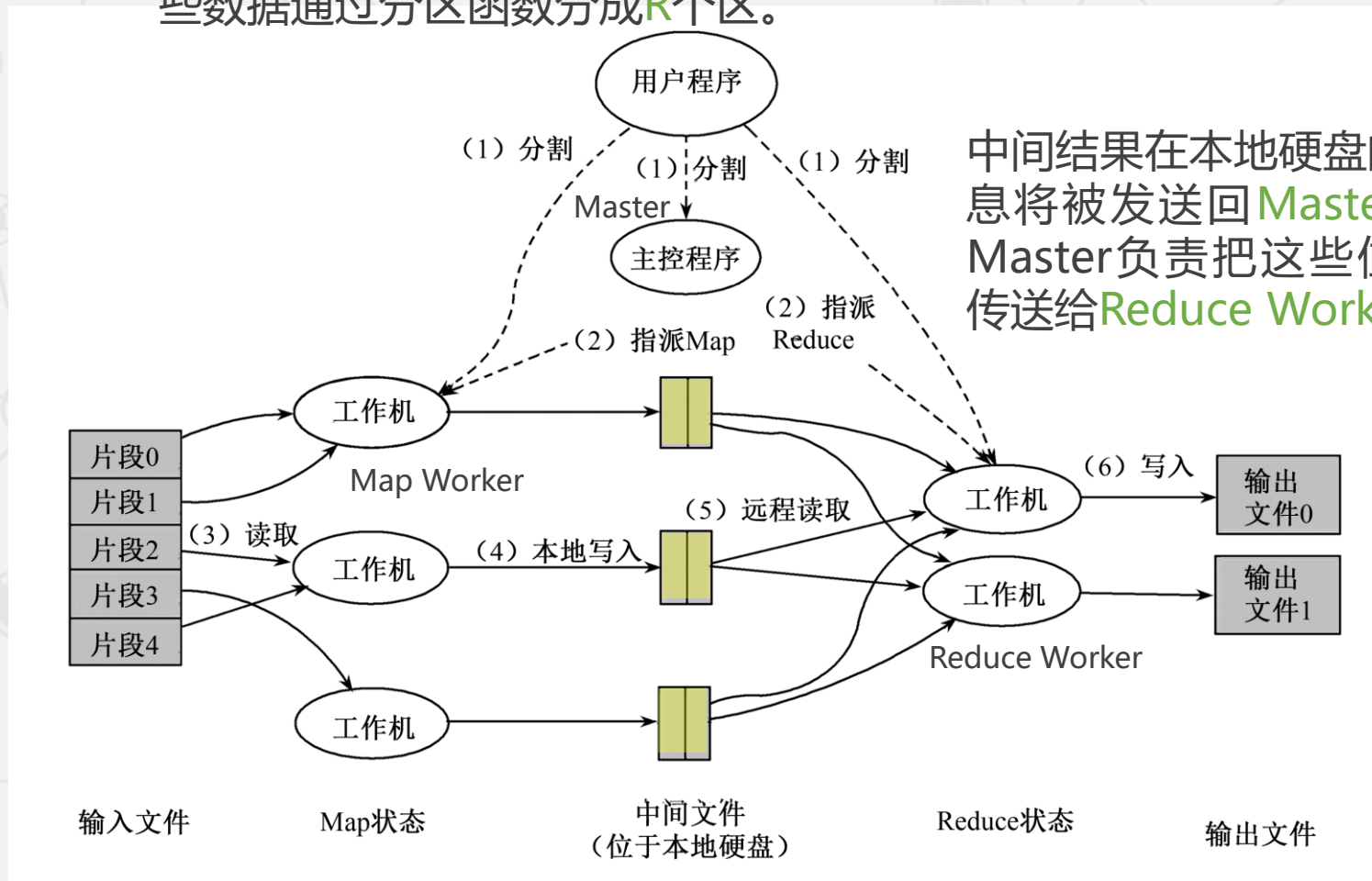
(3) **读取并处理文件块**。一个被分配了Map任务的Worker读取并处理相关的输入块。

它处理输入的数据，并且将分析出的<key, value>对传递给用户定义的Map函数。Map函数产生的中间结果暂时缓冲到内存。



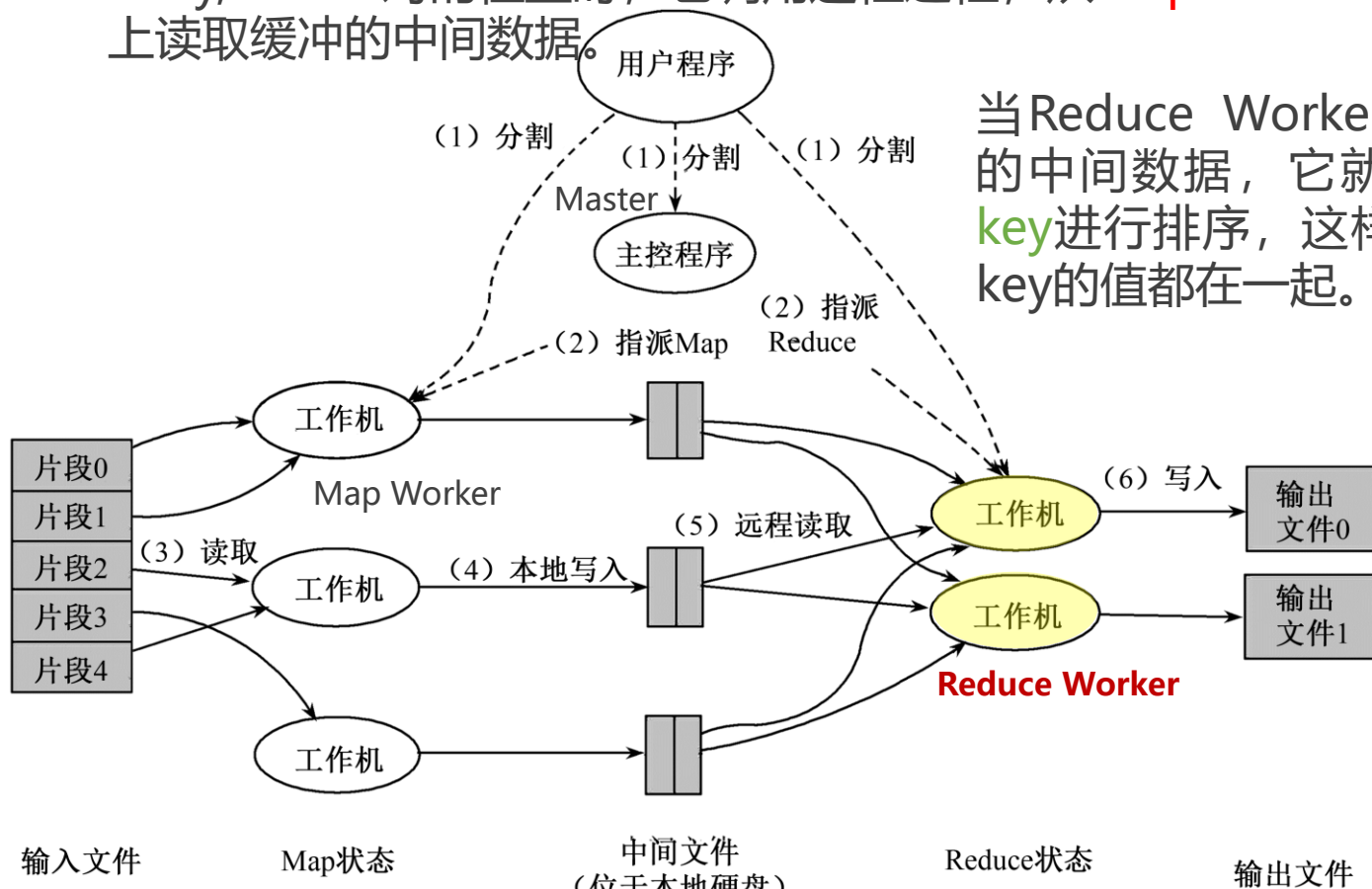
2.2 分布式数据处理 MapReduce

- **实现机制** (4) **本地写入**。这些缓冲到内存的中间结果将被定时写到本地硬盘，这些数据通过分区函数分成R个区。



2.2 分布式数据处理 MapReduce

- **实现机制** (5) **远程读取缓存**。当Master通知执行Reduce的Worker关于中间<key,value>对的位置时，它调用远程过程，从**Map Worker**的本地硬盘上读取缓冲的中间数据。

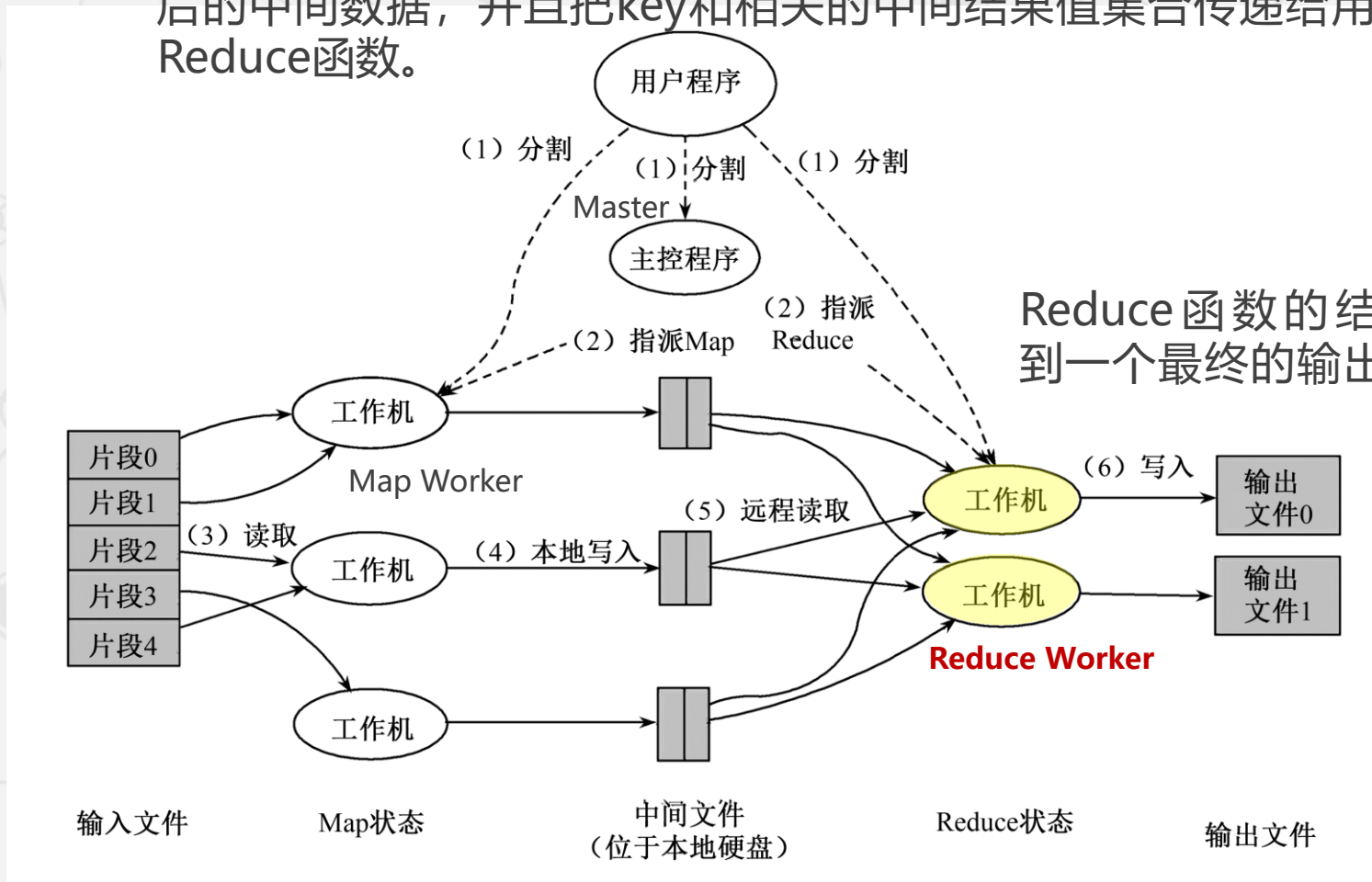


当Reduce Worker读到所有的中间数据，它就使用中间key进行排序，这样可使相同key的值都在一起。

因为有许多不同key的Map都对应相同的Reduce任务，所以，key (Reduce任务) 的排序是必须的。此外，如果中间结果集过于庞大，那么就需要使用外排序。

2.2 分布式数据处理MapReduce

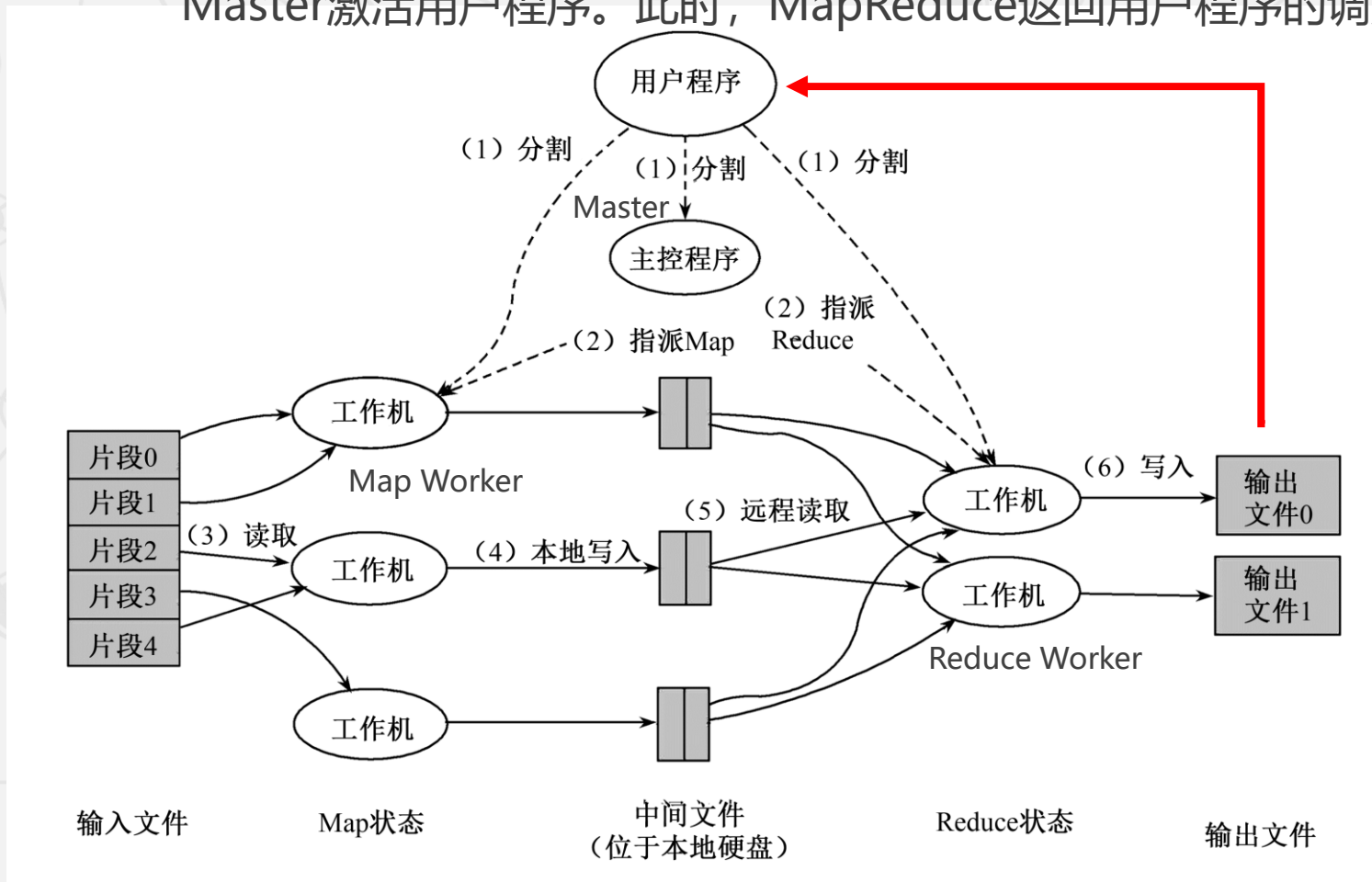
- **实现机制** (6) **写入**。Reduce Worker根据每一个唯一中间key来遍历所有的排序后的中间数据，并且把key和相关的中间结果值集合传递给用户定义的Reduce函数。



2.2 分布式数据处理MapReduce

● 实现机制

(7) **返回**。当所有的Map任务和Reduce任务都完成的时候，Master激活用户程序。此时，MapReduce返回用户程序的调用点。



2.2 分布式数据处理 MapReduce

● 容错机制

由于MapReduce在成百上千台机器上处理海量数据，所以容错机制是不可或缺的。总的来说，MapReduce通过重新执行失效的地方来实现容错。

Master失效

Master会周期性地设置检查点 (checkpoint)，并导出Master的数据。一旦某个任务失效，系统就从最近的一个检查点恢复并重新执行。

由于只有一个Master在运行，如果Master失效了，则只能终止整个MapReduce程序的运行并重新开始。

Worker失效

相对于Master失效，Worker失效算是一种常见状态。

Master会周期性地给Worker发送ping命令，如果没有Worker的应答，则Master认为Worker失效，终止对这个Worker的任务调度，把失效Worker的任务调度到其他Worker上重新执行。

2.2 分布式数据处理 MapReduce

2.2.1 产生背景

2.2.2 编程模型

2.2.3 实现机制

► 2.2.4 案例分析

假设有一批海量的数据，每个数据都是有26个字母组成的字符串，原始数据集是完全无序的。怎样通过MapReduce完成排序工作，使其有序（字典序）呢？

2.2 分布式数据处理 MapReduce

- 第一个步骤

对原始的数据进行分割 (Split) ,
得到N个不同的数据分块。

Split1:	nkklacdedd gfgdfsdfdfd annnbnbvgh
---------	--

Split2:	dfgmdhjydf kghfgcxnkil gjghyotewgbb
---------	--

.....

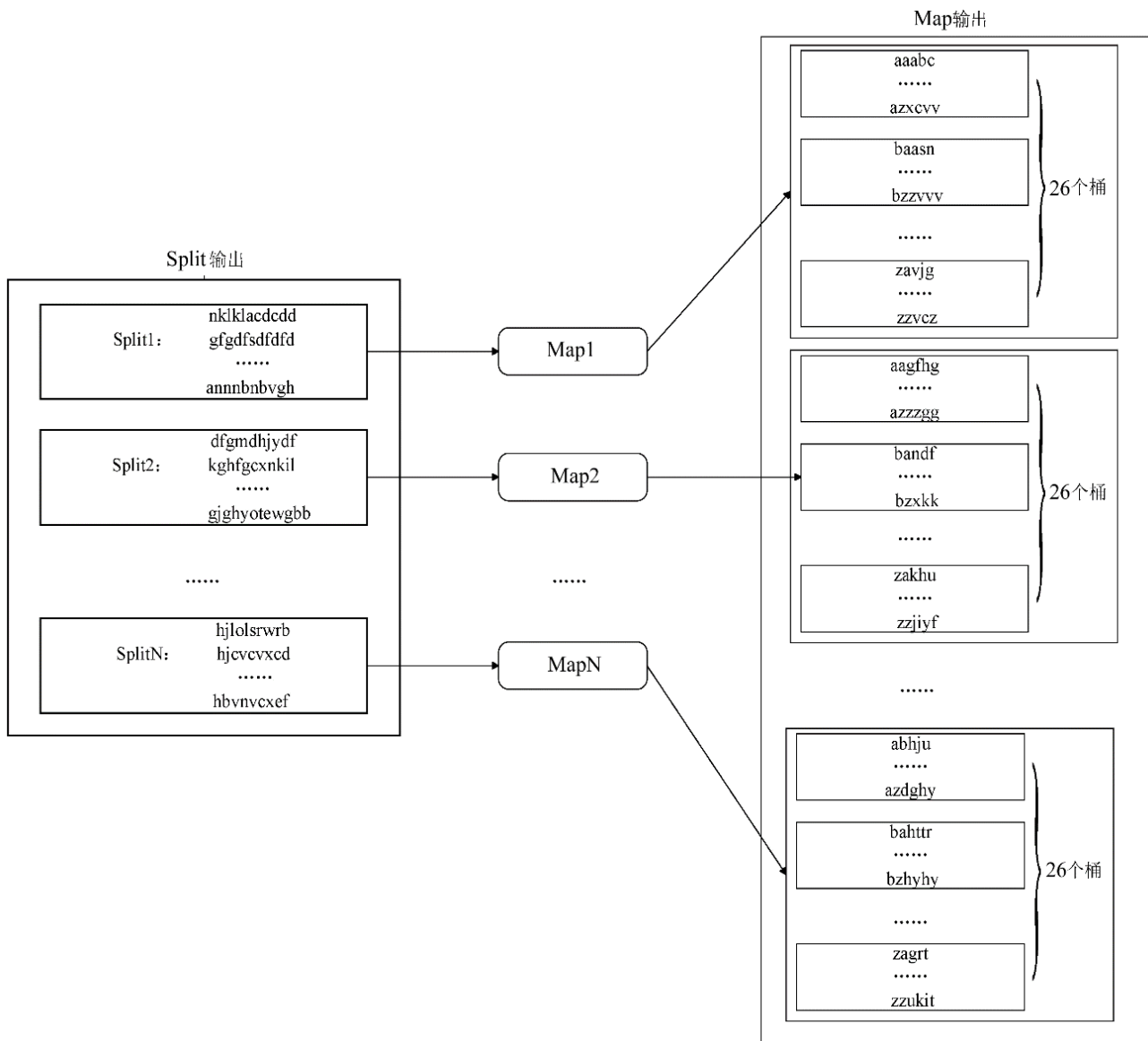
SplitN:	hjlolsrwrbr hjevexced hbnvxcxef
---------	--

2.2 分布式数据处理 MapReduce

• 第二个步骤

对每一个数据分块都启动一个**Map**进行处理。

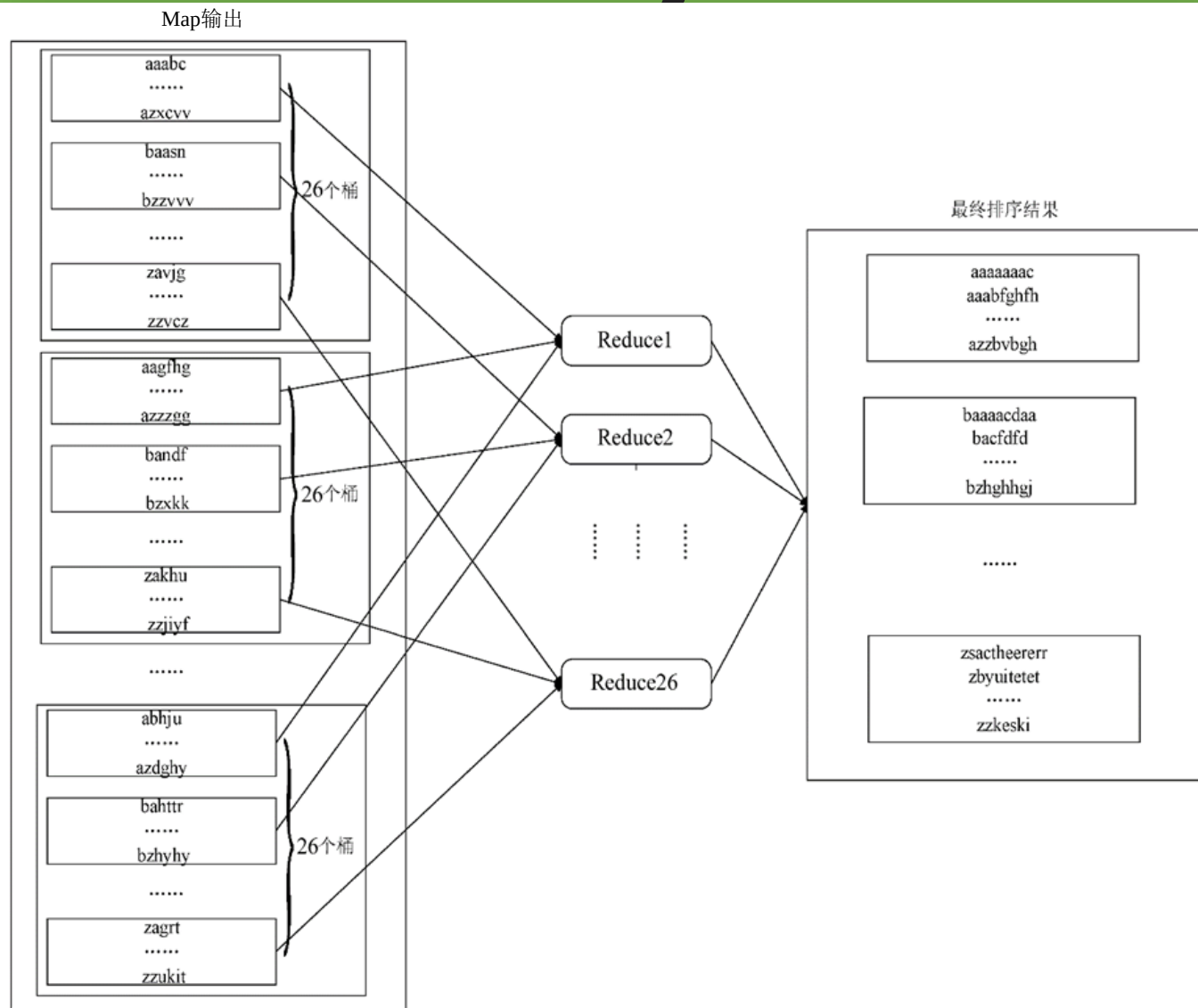
采用**桶排序**的方法，每个Map中按照首字母将字符串分配到**26**个不同的桶中。



2.2 分布式数据处理 MapReduce

• 第三个步骤

对于Map之后得到的中间结果，启动26个Reduce。
按照首字母将Map中不同桶中的字符串集合放置到相应的Reduce中进行处理。

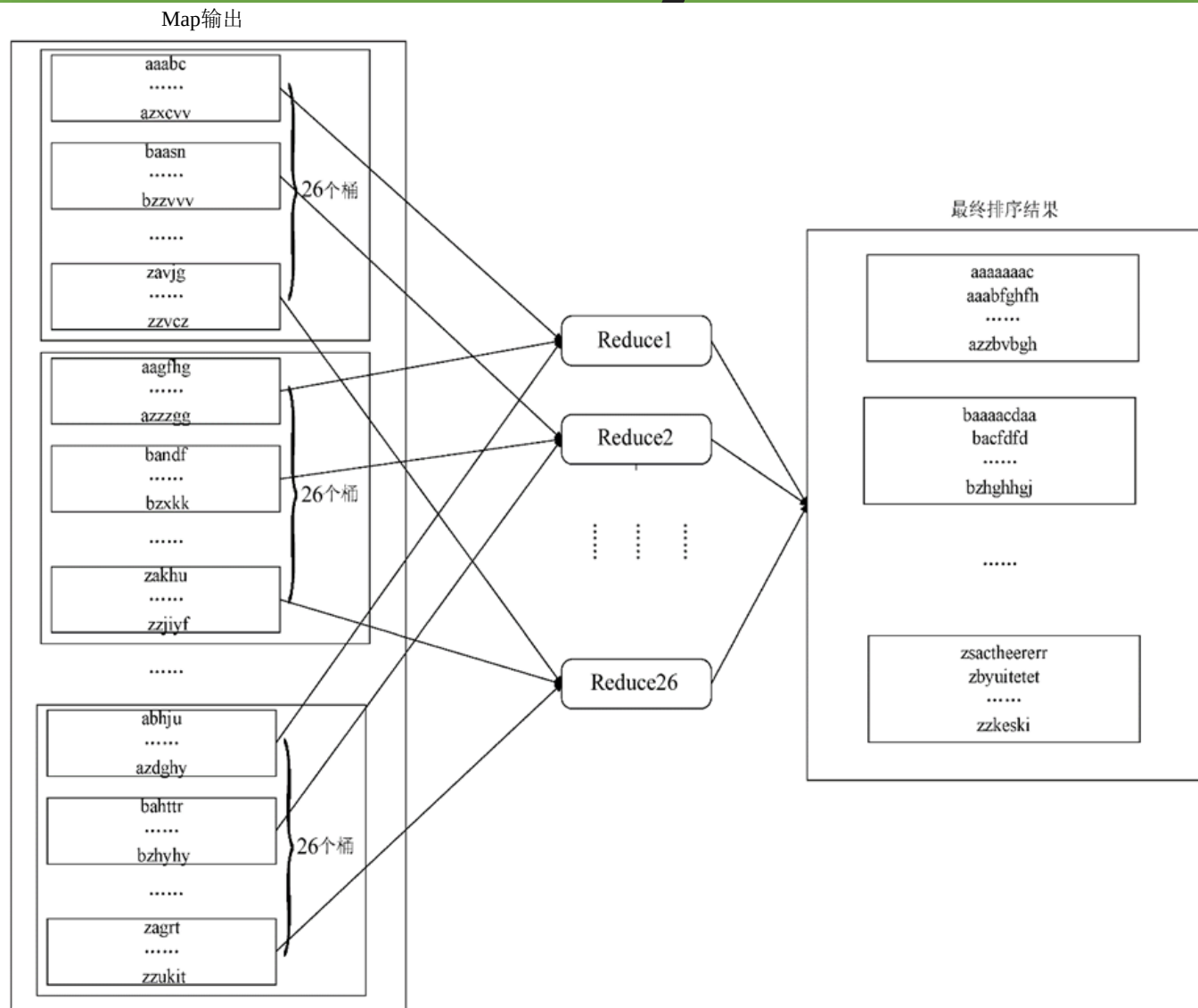


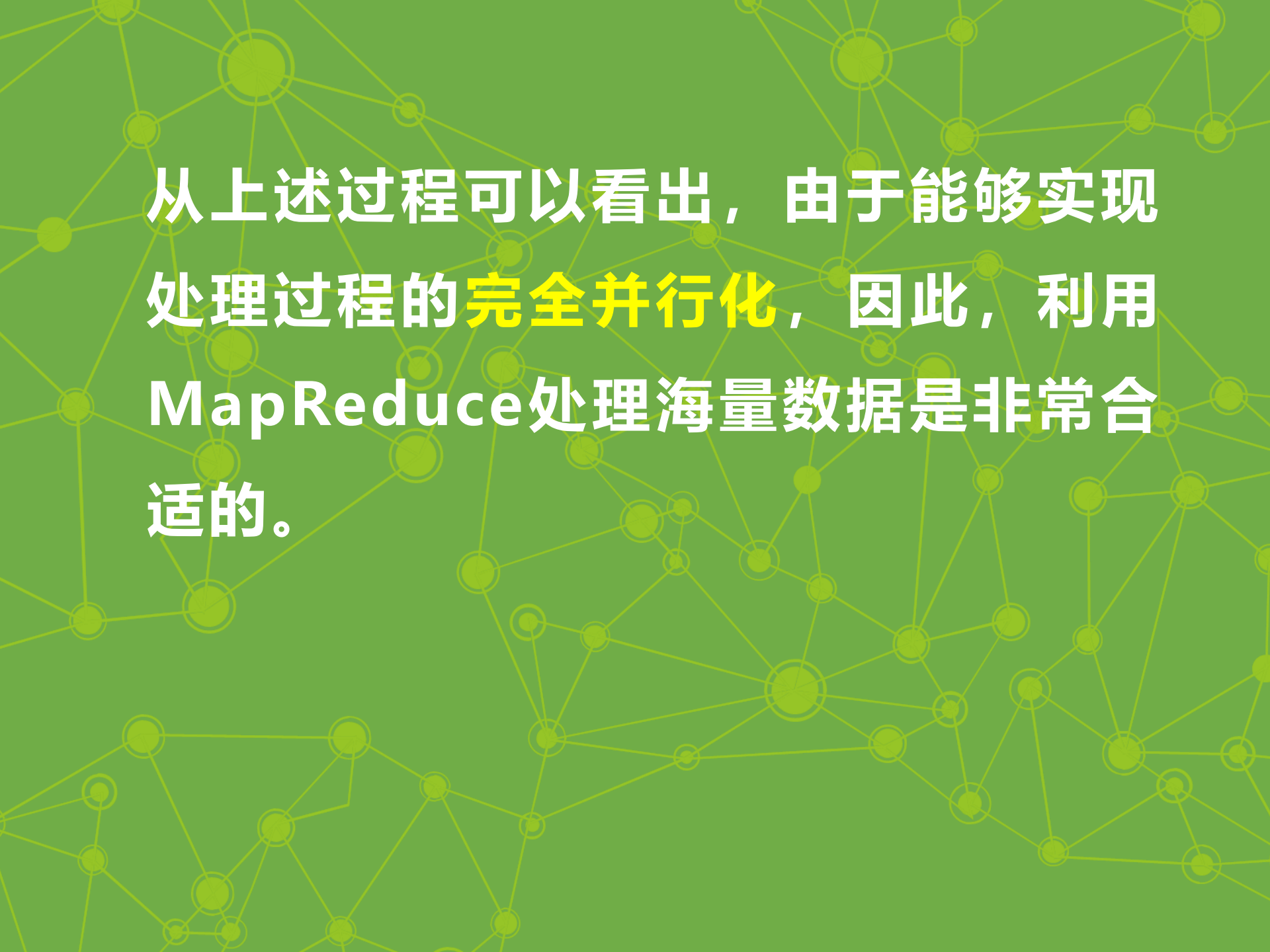
2.2 分布式数据处理 MapReduce

• 第三个步骤

具体来说，就是首字母为a的字符串全部放在Reduce 1中处理，首字母为b的字符串全部放在Reduce 2，以此类推。

每个Reduce对于其中的字符串进行排序，结果直接输出。





从上述过程可以看出，由于能够实现处理过程的**完全并行化**，因此，利用MapReduce处理海量数据是非常合适的。

练习: WordCount

File:

hadoop is very good

mapreduce is very good

hadoop is easy to learn

mapreduce is easy to learn

2.2 分布式数据处理 MapReduce

(1) 分成2个文件

hadoop is very good
mapreduce is very good

hadoop is easy to learn
mapreduce is easy to learn

2.2 分布式数据处理MapReduce

(2) 执行Map任务，存储中间结果。

hadoop is very good
mapreduce is very good

<good, 2>
<hadoop, 1>
<is, 2>
<mapreduce, 1>
<very, 2>

hadoop is easy to learn
mapreduce is easy to learn

<easy, 2>
<hadoop, 1>
<is, 2>
<learn, 2>
<mapreduce, 1>
<to, 2>

2.2 分布式数据处理 MapReduce

(3) 执行Reduce任务，拼接结果。

hadoop is very good
mapreduce is very good

hadoop is easy to learn
mapreduce is easy to learn

<good, 2>
<hadoop, 1>
<is, 2>
<mapreduce, 1>
<very, 2>

<easy, 2>
<hadoop, 1>
<is, 2>
<learn, 2>
<mapreduce, 1>
<to, 2>

<easy, 2>
<good, 2>
<hadoop, 2>
<is, 4>
<learn, 2>
<mapreduce, 2>
<to, 2>
<very, 2>

**练习：计数（将下面的文件划分为2
个子文件进行MapReduce计算）**

File:

2 2 3 5 5 7 8 9 0 1

8 2 9 3 6 4 4 6 7 4

9 5 7 4 5 2 1 1 6 8

6 3 1 1 9 0 0 6 3 0

2.2 分布式数据处理MapReduce

(1) 分成2个文件

```
2 2 3 5 5 7 8 9 0 1  
8 2 9 3 6 4 4 6 7 4
```

```
9 5 7 4 5 2 1 1 6 8  
6 3 1 1 9 0 0 6 3 0
```

2.2 分布式数据处理MapReduce

(2) 执行Map任务，存储中间结果。

2 2 3 5 5 7 8 9 0 1
8 2 9 3 6 4 4 6 7 4

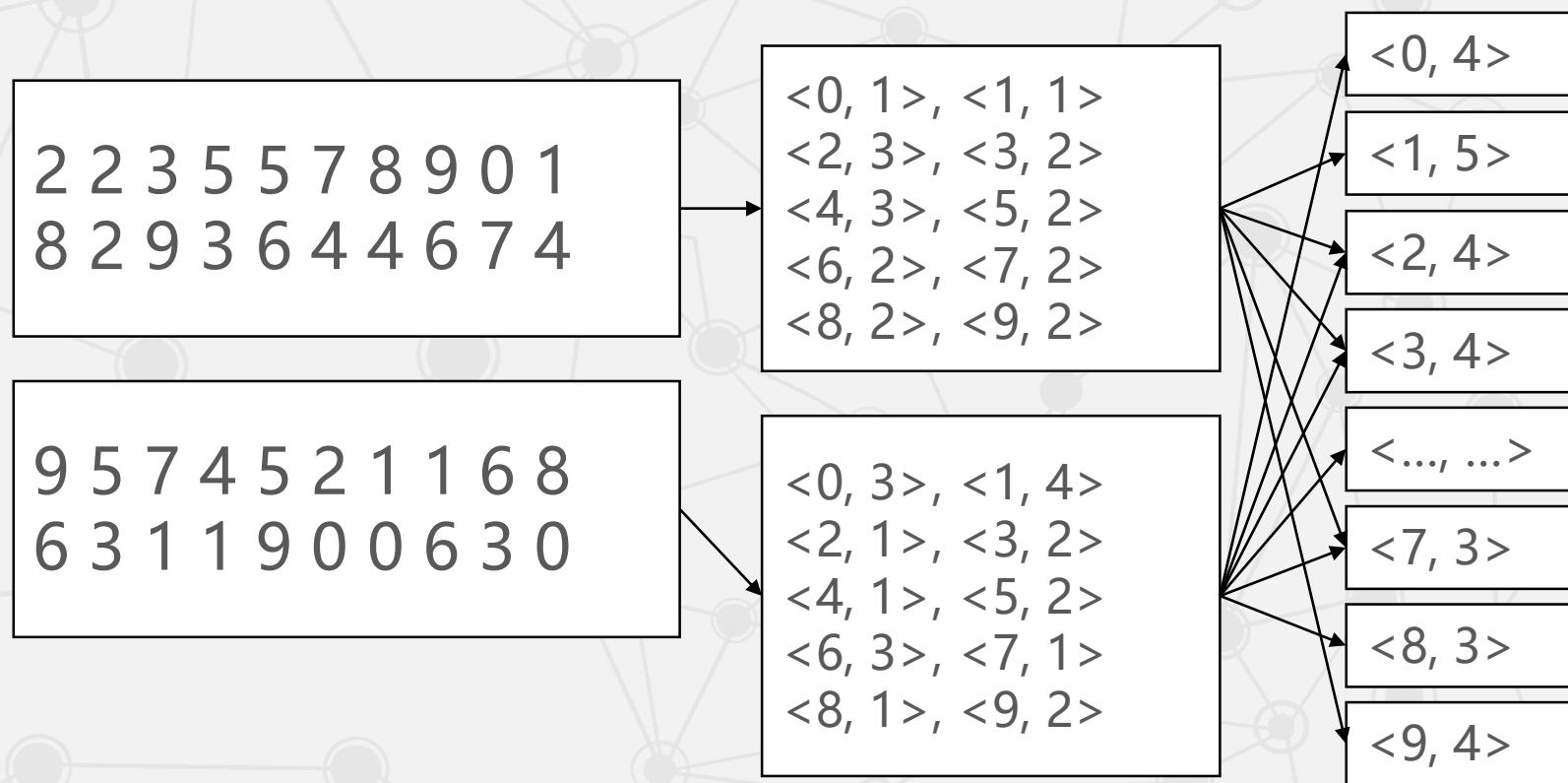
<0, 1>, <1, 1>
<2, 3>, <3, 2>
<4, 3>, <5, 2>
<6, 2>, <7, 2>
<8, 2>, <9, 2>

9 5 7 4 5 2 1 1 6 8
6 3 1 1 9 0 0 6 3 0

<0, 3>, <1, 4>
<2, 1>, <3, 2>
<4, 1>, <5, 2>
<6, 3>, <7, 1>
<8, 1>, <9, 2>

2.2 分布式数据处理 MapReduce

(3) 执行Reduce任务，拼接结果。



The background of the image is a dark gray field filled with a complex network of thin, light gray lines. These lines connect numerous circular nodes of varying sizes, creating a web-like or molecular structure. The nodes are also light gray, and some are larger than others, adding depth to the pattern.

本章未完待续